Blok C (80 min): Ollama, Open WebUI i testovi u Docker Composeu

> 🎯 Ishod vježbe
>
> U osobnom repozitoriju `praktikum-<github-korisnik>` otvoren je **pull request** s grane `feature/compose-ollama`. PR sadrži:
>
> - servis `ollama-server` (slika `ollama/ollama`) s imenovanim volumenom za modele i provjerom zdravlja (`healthcheck`), **bez** objavljenog porta,
> - jednokratni servis `ollama-pull` koji preuzima model i završava,
> - servis `open-webui` (web sučelje na `http://localhost:3000`),
> - korisnički definiranu bridge mrežu `backend` na kojoj su svi servisi, umjesto host mreže; servis `llm` Ollamu traži po DNS imenu kontejnera: `OLLAMA_BASE_URL=http://ollama-server:11434`,
> - `Dockerfile.test`: jednostavni Dockerfile čiji `CMD` pokreće pytest (postojeći unit testovi),
> - servis `tests` u `compose.yaml` koji pokreće unit testove u kratkoživućem kontejneru,
> - `compose.txt` s izlazom provjera iz koraka 8 i snimku zaslona Open WebUI-ja u opisu PR-a.
>
> **Kriteriji prihvaćanja** (svi moraju vrijediti, nastavnik ih provjerava u PR-u):
>
> 1. Nakon `docker compose up -d`: `ollama-server` je `healthy`, `open-webui` radi, a `ollama-pull` je `Exited (0)`.
> 2. `docker compose run --rm llm status` ispisuje `Ollama is running at http://ollama-server:11434`.
> 3. `docker compose run --rm tests` završava s `7 passed` i izlaznim kodom `0`.
> 4. `ollama-server` nema objavljen port: u `docker compose ps` stupac PORTS za njega prikazuje samo `11434/tcp`.
> 5. Nakon `docker compose down` i ponovnog `docker compose up -d` model je i dalje na popisu `docker compose exec ollama-server ollama list` (volumen je preživio).
> 6. Poruka završnog commita i naslov PR-a slijede Conventional Commits i povećavaju **minor** verziju.
>
> **Predaja:** otvoren PR. Ne spajati ga sami; PR je predaja i ostaje otvoren za pregled.

Polazište je rezultat bloka B (grana `feature/compose`). Gotovo rješenje nalazi se u docker-lab/blok-c/; za repozitorij koji je nakon tjedna 3 Python paket u docker-lab/paket/ i u okvirima "Paket".

Sadržaj

0. Grana i slike
1. Servis `ollama-server`: dugoživući kontejner
2. Mreže: zadani bridge, zadana mreža projekta, korisnička mreža
3. Jednokratni servis `ollama-pull` i `depends_on`
4. Unit testovi u kontejneru: `Dockerfile.test`
5. Servis `tests`
6. Open WebUI
7. Volumeni: što preživi `down`
8. Provjere, commit i pull request
9. Pregled: DevOps datoteke u korijenu repozitorija

0. Grana i slike

```
cd ~/praktikum-$(gh api user -q .login)
git fetch origin
git checkout -b feature/compose-ollama origin/feature/compose   # or origin/main if the
block B PR is already merged
```

Ako PR iz bloka B još nije spojen, PR iz ovog bloka otvaramo prema grani `feature/compose` (vidi korak 8).

Slike su velike. Preuzeti ih **kod kuće, prije termina** (na nastavi samo provjeriti `docker images`):

```
docker pull ollama/ollama:0.40.13                  # about 3.4 GB to download
docker pull ghcr.io/open-webui/open-webui:v0.7.2   # about 1.5 GB to download
```

Model (`llama3.2:1b`, oko 1,3 GB) preuzima servis `ollama-pull` u koraku 3. Na sporoj mreži koristiti manji model iz tjedna 2: `export OLLAMA_MODEL=qwen2.5:0.5b` (ili ga upisati u `.env` pokraj `compose.yaml`).

Ollama s hosta (tjedan 2) više nam ne treba; može i dalje raditi, ništa ne smeta jer Ollamu iz Composea ne objavljujemo na portu 11434.

1. Servis `ollama-server`: dugoživući kontejner

Na početak sekcije `services:` u `compose.yaml` dodati:

```
  ollama-server:
    image: ollama/ollama:0.40.13
    volumes:
      - ollama-models:/root/.ollama      # named volume: models survive `down` and
image upgrades
    # No `ports:`: only containers on the same network can reach Ollama, the host
cannot.
    healthcheck:
      test: ["CMD", "ollama", "list"]    # the image has no curl; the ollama CLI asks
the server
      interval: 5s
      timeout: 5s
      retries: 12
      start_period: 5s
    restart: unless-stopped
```

i na kraj datoteke (na razini `services:`, bez uvlake):

```
volumes:
  ollama-models:
```

```
docker compose up -d ollama-server
docker compose ps                          # STATUS: Up ... (health: starting), then
(healthy)
docker compose logs -f ollama-server       # Ctrl+C stops following, not the container
docker compose exec ollama-server ollama list   # a second process in the running
```

```
container
docker volume ls                              # praktikum_ollama-models
```

`ollama-server` je **dugoživući** kontejner: poslužitelj koji radi dok ga ne zaustavimo. Zato ima `restart: unless-stopped` (podiže se nakon pada ili restarta Dockera) i `healthcheck` (Docker periodički provjerava da poslužitelj odgovara, a ne samo da proces postoji). Usporediti s `llm` iz bloka B: kratkoživući kontejner koji obavi jednu naredbu i završi; isto će vrijediti za `tests` (korak 5).

Ollama u kontejneru radi na CPU-u. Na macOS-u Docker nema pristup GPU-u (Metal), a na Windowsima s NVIDIA karticom GPU se može uključiti dodatnom postavkom (`deploy.resources.reservations.devices`), što ovdje preskačemo. Mali model (`llama3.2:1b`, `qwen2.5:0.5b`) je i na CPU-u dovoljno brz.

2. Mreže: zadani bridge, zadana mreža projekta, korisnička mreža

Servis `llm` je još na host mreži i traži Ollamu na `localhost`. Ollama je sada u kontejneru i nema objavljen port, pa ćemo ga **prenamijeniti** da je traži po imenu servisa, `ollama-server`.

a) Zadani bridge (`bridge`, `docker0`). U servisu `llm` zamijeniti `network_mode: host` s `network_mode: bridge` i promijeniti URL:

```
    network_mode: bridge
    environment:
      OLLAMA_BASE_URL: http://ollama-server:11434   # container DNS name instead of
localhost
```

```
docker compose run --rm llm status; echo "exit code: $?"
# Ollama is NOT running at http://ollama-server:11434, exit code: 1
```

Zadani bridge je Dockerova najstarija mreža i **nema DNS po imenu** (isto smo vidjeli u tjednu 3 s `nginx`). Osim toga, `ollama-server` uopće nije na njoj.

b) Zadana mreža projekta. Obrisati liniju `network_mode: bridge`.

```
docker compose run --rm llm status                      # Ollama is running at
http://ollama-server:11434
docker network ls                                       # praktikum_default
docker network inspect praktikum_default --format '{{range .Containers}}{{.Name}}
{{.IPv4Address}}{{println}}{{end}}'
docker compose run --rm --entrypoint python llm -c "import socket;
print(socket.gethostbyname('ollama-server'))"
```

Servis bez `network_mode` i bez `networks` Compose stavlja na mrežu `<projekt>_default`. To je **korisnički definirana bridge mreža**: ima ugrađeni DNS (`127.0.0.11`) koji ime servisa razrješava u IP adresu kontejnera. Zato je URL `http://ollama-server:11434`, a ne IP adresa koja se mijenja pri svakom pokretanju.

c) Imenovana korisnička mreža. Mrežu imenujemo izričito, da se iz `compose.yaml` vidi tko s kim razgovara. Na kraj datoteke dodati:

```
networks:
  backend:                              # user-defined bridge: DNS by service name,
isolated from other projects
```

i u **svaki** servis (`ollama-server`, `llm`) dodati:

```
    networks: [backend]
```

```
docker compose down                     # removes praktikum_default
docker compose up -d ollama-server
docker network ls                       # praktikum_backend
```

Način	U `compose.yaml`	DNS po imenu servisa	Kada
host	`network_mode: host`	ne (sve je `localhost`)	alat treba servis koji radi na hostu (blok B)
zadani bridge	`network_mode: bridge`	ne	praktički nikad u Composeu
zadana mreža projekta	ništa	da	mali projekti
korisnička mreža	`networks: [backend]` + `networks:` na kraju	da	izričito: tko s kim smije razgovarati

3. Jednokratni servis `ollama-pull` i `depends_on`

Model treba preuzeti jednom, prije nego ga alat koristi. To je posao za **jednokratni** kontejner: ista slika `ollama/ollama`, ali samo CLI, koji kaže poslužitelju `ollama-server` da preuzme model i završi.

```
  ollama-pull:
    # One-shot job: same image, but only the CLI, which pulls the model into ollama-
server and exits.
    image: ollama/ollama:0.40.13
    entrypoint: ["ollama", "pull", "${OLLAMA_MODEL:-llama3.2:1b}"]
    environment:
      OLLAMA_HOST: http://ollama-server:11434   # where the ollama CLI sends its
requests
    networks: [backend]
    depends_on:
      ollama-server:
        condition: service_healthy
    restart: "no"
```

U servis `llm` dodati ovisnost:

```
    depends_on:
      ollama-pull:
        condition: service_completed_successfully
```

Uvjet u `depends_on`	Compose čeka da ovisnost…
`service_started` (zadano)	bude pokrenuta (ne znači da je spremna)
`service_healthy`	prođe `healthcheck`
`service_completed_successfully`	završi s izlaznim kodom 0

```
docker compose up -d
docker compose ps -a        # ollama-server (healthy), ollama-pull Exited (0)
docker compose logs ollama-pull
docker compose exec ollama-server ollama list   # the model is now in the named volume
```

Varijabla `OLLAMA_HOST` je Ollamina vlastita (adresa na kojoj CLI traži poslužitelj); naš alat koristi `OLLAMA_BASE_URL` (tjedan 2). Sada alat ima i poslužitelj i model:

```
docker compose run --rm llm ask "What is a container network?"   # the model is there,
the CLI answers
```

Lanac ovisnosti. Backend je dio orkestracije, pa Compose osigurava da radi prije alata:

```
llm ──(service_completed_successfully)──▶ ollama-pull ──(service_healthy)──▶ ollama-
server
```

Isprobati od nule, bez ičega pokrenutog:

```
docker compose down
docker compose run --rm llm status
# shortened output:
# Container praktikum-ollama-server-1  Started
# Container praktikum-ollama-server-1  Waiting
# Container praktikum-ollama-server-1  Healthy
# Container praktikum-ollama-pull-1    Started
# Container praktikum-ollama-pull-1    Exited
# Ollama is running at http://ollama-server:11434
```

Compose je pokrenuo backend, pričekao `healthcheck`, pustio `ollama-pull` da preuzme model i tek onda pokrenuo alat. Zašto `service_healthy`, a ne samo `service_started`? Kontejner koji je pokrenut nije nužno spreman: Ollami treba nekoliko sekundi da otvori port. `healthcheck` (`ollama list`) prolazi tek kad poslužitelj zaista odgovara.

	Ollama na hostu (blok B)	Ollama u Composeu (sada)
Gdje je backend	na računalu, izvan `compose.yaml`	servis `ollama-server` u istom `compose.yaml`
Tko ga pokreće	mi, ručno	Compose, prije alata
Kako alat zna da je spreman	ne zna; ako ne radi, `status` javlja grešku	`depends_on` + `condition: service_healthy` (i `healthcheck`)
Model	`ollama pull` ručno	servis `ollama-pull`

4. Unit testovi u kontejneru: `Dockerfile.test`

Postojećih 7 testova (tjedni 2 i 3) su **unit testovi**: `requests.request` je zamijenjen lažnom funkcijom, pa provjeravaju naš kod izolirano i prolaze bez Ollame. Lokalno, u virtualnom okruženju:

```
python -m pytest -v        # 7 passed
```

Sada ih pokrećemo i u kontejneru, da rezultat ne ovisi o tome što je instalirano na čijem računalu. Testove pokrećemo **kad se kontejner pokrene** (`CMD`), a ne dok se slika gradi (`RUN`, kao faza `test` u tjednu 3): slika `praktikum-llm` ostaje bez pytesta, a rezultat testova je izlazni kod kontejnera.

Pišemo ga kao jednostavni Dockerfile u jednoj fazi, kao `Dockerfile` iz bloka B. Nova datoteka `Dockerfile.test`:

```
# Simple, single-stage image that runs the unit tests when the container STARTS.
FROM python:3.13-alpine

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

# Dependencies first: this layer is rebuilt only when the requirements change.
COPY requirements.txt requirements-dev.txt ./
RUN pip install --no-cache-dir -r requirements-dev.txt

# Code and tests last: they change most often.
COPY version.txt ollama_client.py llm.py ./
COPY tests/ tests/

# The exit code of the container is the exit code of pytest: 0 = all passed, 1 = a test
failed.
CMD ["python", "-m", "pytest", "-v"]
```

	`Dockerfile`	`Dockerfile.test`
Ovisnosti	`requirements.txt`	`requirements-dev.txt` (+ `pytest`)
Kopira `tests/`	ne	da
Što radi kad se pokrene	`ENTRYPOINT ["python", "llm.py"]`	`CMD ["python", "-m", "pytest", "-v"]`
Rezultat	ispis alata	izlazni kod pytesta

> **Paket.** Umjesto `requirements*.txt` i modula kopirati `pyproject.toml`, `version.txt` i `src/` pa `pip install ".[dev]"`; gotova datoteka.

5. Servis `tests`

U `compose.yaml` dodati servis `tests`:

```
  tests:
    build:
      context: .
      dockerfile: Dockerfile.test  # simple single-stage image whose CMD runs pytest
    image: praktikum-llm-tests
    profiles: [test]               # not started by `docker compose up`, only by
`docker compose run tests`
```

```
docker compose run --rm --build tests; echo "exit code: $?"   # 7 passed, exit code 0
docker compose ps -a                                          # no tests container: it
is short-lived and removed
```

Servis nema `depends_on`, `networks` ni `environment`: unit testovi ne razgovaraju s Ollamom, pa im backend ne treba. Usporediti od nule:

```
docker compose down
docker compose run --rm tests        # only the tests container starts; ollama-server
stays down
docker compose run --rm llm status   # the llm service starts the whole chain first
(step 3)
```

Rezultat kratkoživućeg kontejnera je njegov **izlazni kod**. Isprobati da test padne: u `tests/test_version.py` privremeno zamijeniti očekivani `"0.0.0"` s `"0.0.1"` i ponoviti `docker compose run --rm --build tests; echo "exit code: $?"` (`1 failed, 6 passed`, izlazni kod `1`), pa vratiti datoteku (`git checkout tests/test_version.py`). Na izlaznom kodu će CI pipeline u tjednu 5 odlučiti je li build zelen.

Servis	Vrsta	Treba backend	Rezultat
`ollama-server`	dugoživući	–	radi dok ga ne zaustavimo (`healthy`)
`ollama-pull`	jednokratni	`ollama-server` (`service_healthy`)	izlazni kod 0: model je preuzet
`llm`	kratkoživući	`ollama-pull` (`service_completed_successfully`)	ispis alata
`tests`	kratkoživući	ne	izlazni kod pytesta

Primijetiti: `docker compose build` (bez imena servisa) gradi samo servise koji nisu u profilima, a `llm` i `tests` jesu. Gradi se s `docker compose --profile cli --profile test build` ili, kao gore, `docker compose run --build`.

6. Open WebUI

Open WebUI je web sučelje za razgovor s modelima, slično ChatGPT-u, koje razgovara s Ollamom preko istog REST API-ja kao naš alat.

```
  open-webui:
    image: ghcr.io/open-webui/open-webui:v0.7.2
    ports:
      - "127.0.0.1:3000:8080"           # http://localhost:3000, only from this
machine
    environment:
      OLLAMA_BASE_URL: http://ollama-server:11434
      WEBUI_AUTH: "False"               # single-user local setup; never on a shared
server
    volumes:
      - open-webui-data:/app/backend/data
    networks: [backend]
    depends_on:
      ollama-server:
```

```
        condition: service_healthy
    restart: unless-stopped
```

U `volumes:` na kraju datoteke dodati `open-webui-data:`.

```
docker compose up -d
docker compose logs -f open-webui        # first start takes a minute; wait for
"Uvicorn running on http://0.0.0.0:8080"
```

Otvoriti `http://localhost:3000`, odabrati model i postaviti pitanje. Snimku zaslona staviti u opis PR-a.

Dva detalja s porta: `3000:8080` znači *port 3000 na hostu → port 8080 u kontejneru*, a `127.0.0.1:` ispred znači da sučelje vidi samo ovo računalo, ne i ostali na istoj Wi-Fi mreži. `WEBUI_AUTH: "False"` isključuje prijavu; to je u redu samo lokalno. Na zajedničkom poslužitelju (tema 6) prijava mora biti uključena.

Primijetiti da `ollama-server` i dalje nema `ports:`: preglednik razgovara s Open WebUI-jem, a Open WebUI s Ollamom unutar mreže `backend`.

7. Volumeni: što preživi down

```
docker compose down                      # containers and the network are removed
docker volume ls                         # praktikum_ollama-models and praktikum_open-
webui-data are still there
docker compose up -d
docker compose exec ollama-server ollama list    # the model is still there: nothing is
downloaded again
```

Vrsta	U `compose.yaml`	Gdje su podaci	Primjer
bind mount	`./:/app`	direktorij na hostu koji sami biramo	kod u razvoju (blok B, korak 3)
imenovani volumen	`ollama-models:/root/.ollama` + `volumes:` na kraju	prostor kojim upravlja Docker	modeli, baza Open WebUI-ja

`docker compose down -v` briše i imenovane volumene (model bi se preuzimao ponovno). Ne pokretati ga bez razloga.

8. Provjere, commit i pull request

```
{
  echo "## up";       docker compose up -d >/dev/null 2>&1; docker compose ps -a --
format '{{.Service}}: {{.Status}} {{.Ports}}'
  echo "## status";   docker compose run --rm llm status 2>/dev/null
  echo "## tests";    out=$(docker compose run --rm --build tests 2>&1); echo "exit
code: $?"; echo "$out" | tail -1
  echo "## persist";  docker compose down >/dev/null 2>&1; docker compose up -d
>/dev/null 2>&1
                      docker compose exec ollama-server ollama list
} | tee compose.txt
```

(Ako se `open-webui` u ispisu još ne vidi kao `Up`, pričekati minutu i ponoviti.)

Poruka završnog commita. Kao u bloku B: završni commit povećava **minor** verziju prema Conventional Commits; prefiks pronađite sami, u naredbama piše `<type>`.

```
git add compose.yaml Dockerfile.test compose.txt
git status                        # .env and .venv must NOT be listed
git commit -m "<type>: add Ollama, Open WebUI and tests to Docker Compose"
git push -u origin HEAD
gh pr create --base feature/compose --title "<type>: add Ollama, Open WebUI and tests
to Docker Compose" --body "Week 4, block C. Checks: see compose.txt"
```

(`--base main` ako je PR iz bloka B već spojen.) Nakon `gh pr create` dodati snimku zaslona Open WebUI-ja u opis PR-a na GitHubu.

Predaja (praktični zadatak 4, dio C): otvoren PR iz `feature/compose-ollama`, s datotekama iz ishoda, `compose.txt` koji potvrđuje kriterije 1–5 i snimkom zaslona u opisu (kriterij 6 provjerava se iz poruke commita i naslova PR-a). PR ne spajati sami.

Na kraju, da ne troši memoriju: `docker compose down` (volumeni ostaju za sljedeći put).

9. Pregled: DevOps datoteke u korijenu repozitorija

Nakon četiri tjedna u korijenu `praktikum-<github-korisnik>` uz kod (`llm.py`, `ollama_client.py`, `tests/`) stoji niz datoteka koje ne mijenjaju što alat radi, nego **kako se gradi, testira, pokreće i objavljuje**. To su DevOps datoteke:

Datoteka	Čemu služi	Tko je čita	U gitu	U slici
`.gitignore`	što git ne prati: `.venv/`, `.env`, `__pycache__/`, `.pytest_cache/`	git	da	ne
`.dockerignore`	što ne ulazi u build context: `.git`, `.github`, `.env`, `.venv`, izlazi provjera	`docker build`	da	ne
`.env.example`	predložak varijabli okoline (`OLLAMA_BASE_URL`, `OLLAMA_MODEL`), bez tajni	ljudi: kopiraju ga u `.env`	da	ne
`.env`	lokalne vrijednosti i tajne	`python-dotenv` lokalno; `docker compose` za `${...}` u `compose.yaml`	**ne**	**ne**
`requirements.txt`	ovisnosti za izvođenje, s točnim verzijama	`pip`, `Dockerfile`, `Dockerfile.test`	da	da
`requirements-dev.txt`	`requirements.txt` + alati za razvoj i testove (`pytest`)	`pip` u venvu, `Dockerfile.test`	da	samo u slici za testove

Datoteka	Čemu služi	Tko je čita	U gitu	U slici
`version.txt`	trenutna verzija alata	release-please (povećava je), `llm --version`	da	da
`CHANGELOG.md`	popis promjena po verzijama	release-please (piše ga), ljudi	da	ne
`.github/workflows/release_please.yml`	GitHub Actions: nakon pusha u `main` otvara i ažurira release PR	GitHub Actions	da	ne
`check_env.sh`	provjera lokalnog okruženja (alati, datoteke)	student, lokalno	da	ne
`Dockerfile`	recept slike alata `praktikum-llm` (jedna faza od bloka B)	`docker build`, servis `llm` u Composeu	da	ne
`Dockerfile.test`	recept slike koja u `CMD` pokreće pytest	servis `tests` u Composeu	da	ne
`compose.yaml`	servisi, mreža `backend`, volumeni, ovisnosti i profili cijelog sustava	`docker compose`	da	ne
`docker.txt`, `compose.txt`, `storage.txt`	izlaz provjera, dokaz kriterija za predaju	nastavnik u PR-u	da	ne (u `.dockerignore`)

Stupac "U slici" pokazuje zašto postoji `.dockerignore`: od svih DevOps datoteka u sliku alata ulaze samo `requirements.txt` i `version.txt`, a sve ostalo je konfiguracija **oko** slike. `.env` je jedina datoteka koja nije ni u gitu ni u slici: konfiguracija i tajne dolaze iz okoline.

> **Paket.** `pyproject.toml` zamjenjuje `requirements.txt` i `requirements-dev.txt`: ovisnosti, dodatak `[dev]`, naredbu `llm`, verziju iz `version.txt` i postavke pytesta (`[tool.pytest.ini_options]`). Čitaju ga `pip`, `setuptools` i `pytest`, i ulazi u sliku. U `.dockerignore` i `.gitignore` dodatno stoje `*.egg-info`, `build/` i `dist/`.

U tjednu 5 u `.github/workflows/` dolazi novi workflow koji pokreće `docker compose run --rm tests` na svakom pull requestu i iz izlaznog koda odlučuje je li provjera zelena.

Česti problemi

Simptom	Rješenje
`Ollama is NOT running at http://ollama-server:11434`	Servis `llm` nije na istoj mreži kao `ollama-server`: ukloniti `network_mode`, dodati `networks: [backend]`
`ERROR: file or directory not found: tests` ili `no tests ran`	`Dockerfile.test` nije kopirao testove: `COPY tests/ tests/`

Simptom	Rješenje
`No services to build`	`llm` i `tests` su u profilima: `docker compose --profile cli --profile test build` ili `docker compose run --build <servis>`
`service "ollama-pull" didn't complete successfully: exit 1`	`docker compose logs ollama-pull`. Najčešće nema mreže prema registru modela ili je ime modela krivo (`OLLAMA_MODEL`)
`docker compose up -d --wait` završi s greškom `container praktikum-ollama-pull-1 exited (0)`	`--wait` očekuje da svi pokrenuti kontejneri rade, a `ollama-pull` namjerno završava. Koristiti `docker compose up -d` bez `--wait`
`dependency failed to start: container praktikum-ollama-server-1 is unhealthy`	Poslužitelj se nije podigao na vrijeme: `docker compose logs ollama-server`; na sporom računalu povećati `retries`
`Bind for 127.0.0.1:3000 failed: port is already allocated`	Na hostu nešto već sluša na 3000: promijeniti lijevi port, npr. `"127.0.0.1:3001:8080"`
Open WebUI ne prikazuje modele	`OLLAMA_BASE_URL` u servisu `open-webui` mora biti `http://ollama-server:11434`, a servis na mreži `backend`; zatim `docker compose up -d open-webui`
Odgovori su vrlo spori	Ollama u kontejneru radi na CPU-u; koristiti `OLLAMA_MODEL=qwen2.5:0.5b`
Disk je pun	`docker system df`; stare slike: `docker image prune`; nikako `docker volume prune` ako ne želite ponovno preuzimati modele